Within your application, you will remove most of the restrictions, so you can use simple passwords for testing. Apply the following options, which eliminate the need to use special characters.

Listing 9.15 Configuring password options

```
builder.Services.AddDefaultIdentity<CityBreaksUser>(options =>{
    options.Password.RequireDigit = false;
    options.Password.RequireLowercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequireUppercase = false;
}).AddEntityFrameworkStores<CityBreaksContext>();
```

NOTE If you change the `RequiredLength` value, you will also need to update the `StringLength` validation that has been applied to the `UserName` property in the `RegisterModel` used in the Identity UI registration form. You will get to that a bit later when you scaffold and customize the UI itself.

CONFIGURING SIGNINOPTIONS

The `SignInOptions` enables configuration of various account confirmation requirements, detailed in table 9.4. The `IUserConfirmation<TUser>` interface is provided to enable you to set your own conditions for allowing users to sign in. I don't cover this in any detail here, but the interface has one member: `IsConfirmedAsync`, which returns a `Task<bool>`, wherein you apply your custom logic to determine whether the user is permitted to sign in. You then register your implementation as a service in line with the guidance provided in chapter 7.

Table 9.4 The `SignInOptions` properties